

Universidade de Brasília
Faculdade de Tecnologia

Cálculo Numérico Aplicado

Programa para Casa #5 (PPC5)

Prof. Dr. Rafael Gabler Gontijo

Mateus Leal Silva

Matrícula: 221028134

Brasília, 17 de junho de 2026

Sumário

1	Introdução	2
2	Formulação matemática	2
3	Método numérico implementado	3
3.1	Integração por Runge–Kutta de quarta ordem	3
3.2	Atualização do parâmetro de tiro	3
4	Arquivos de saída do programa	3
5	Parâmetros utilizados na execução de teste	4
6	Resultados numéricos	4
6.1	Convergência do Método do Tiro	4
6.2	Perfis de similaridade	4
6.3	Verificação da condição distante da parede	6
6.4	Coefficiente local de atrito	6
6.5	Determinação de η_{99} e da espessura de camada limite	7
7	Comparação com o valor clássico de $f''(0)$	7
8	Fontes de erro numérico	8
9	Código-fonte Python	8
10	Conclusão	12

1 Introdução

Este relatório apresenta a implementação computacional da solução numérica da equação de Blasius utilizando dois ingredientes principais: o **Método do Tiro**, para transformar o problema de valor de contorno em uma sequência de problemas de valor inicial, e o **método de Runge–Kutta de quarta ordem**, para integrar o sistema de equações diferenciais ordinárias.

A equação de Blasius é dada por

$$f''' + \frac{1}{2}f f'' = 0, \quad (1)$$

com condições de contorno

$$f(0) = 0, \quad f'(0) = 0, \quad f'(\infty) = 1. \quad (2)$$

O objetivo do programa é determinar numericamente o valor de

$$f''(0), \quad (3)$$

gerar os perfis de similaridade $f(\eta)$, $f'(\eta)$ e $f''(\eta)$, estimar η_{99} e calcular grandezas associadas à camada limite laminar.

2 Formulação matemática

Para reduzir a Equação (1) a um sistema de primeira ordem, definem-se as variáveis auxiliares

$$y_1 = f, \quad y_2 = f', \quad y_3 = f''. \quad (4)$$

Assim,

$$\boxed{\begin{aligned} \frac{dy_1}{d\eta} &= y_2, \\ \frac{dy_2}{d\eta} &= y_3, \\ \frac{dy_3}{d\eta} &= -\frac{1}{2}y_1y_3. \end{aligned}} \quad (5)$$

As condições iniciais do problema de valor inicial equivalente são

$$y_1(0) = 0, \quad y_2(0) = 0, \quad y_3(0) = s, \quad (6)$$

em que

$$s = f''(0) \quad (7)$$

é o parâmetro ajustado pelo Método do Tiro.

A função erro usada para corrigir o chute é

$$E(s) = y_2(\eta_{\max}; s) - 1. \quad (8)$$

A convergência é atingida quando

$$|E(s)| < \varepsilon, \quad (9)$$

em que ε é a tolerância escolhida.

3 Método numérico implementado

3.1 Integração por Runge–Kutta de quarta ordem

Para um passo $h = \Delta\eta$, o programa calcula os coeficientes de Runge–Kutta de quarta ordem para as três variáveis do sistema. Em notação compacta, seja

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix}, \quad \mathbf{F}(\mathbf{y}) = \begin{bmatrix} y_2 \\ y_3 \\ -\frac{1}{2}y_1y_3 \end{bmatrix}. \quad (10)$$

Então,

$$\mathbf{k}_1 = h\mathbf{F}(\mathbf{y}_i), \quad (11)$$

$$\mathbf{k}_2 = h\mathbf{F}\left(\mathbf{y}_i + \frac{\mathbf{k}_1}{2}\right), \quad (12)$$

$$\mathbf{k}_3 = h\mathbf{F}\left(\mathbf{y}_i + \frac{\mathbf{k}_2}{2}\right), \quad (13)$$

$$\mathbf{k}_4 = h\mathbf{F}(\mathbf{y}_i + \mathbf{k}_3), \quad (14)$$

com atualização

$$\mathbf{y}_{i+1} = \mathbf{y}_i + \frac{1}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4). \quad (15)$$

3.2 Atualização do parâmetro de tiro

O parâmetro s foi atualizado por Newton–Raphson com derivada numérica:

$$s_{m+1} = s_m - \frac{E(s_m)}{E'(s_m)}, \quad (16)$$

com

$$E'(s_m) \approx \frac{E(s_m + \Delta s) - E(s_m)}{\Delta s}. \quad (17)$$

No programa, Δs foi definido de forma proporcional ao chute corrente, com um valor mínimo para evitar cancelamento numérico:

$$\Delta s = \max(10^{-6}, 10^{-5}|s_m|). \quad (18)$$

4 Arquivos de saída do programa

O programa gera os arquivos listados na Tabela 1.

Tabela 1: Arquivos gerados pelo programa.

Arquivo	Conteúdo
blasius_solution.dat	Tabela com η , f , f' e f''
shooting_log.dat	Histórico de iterações do Método do Tiro
perfil_f.png	Gráfico do perfil $f(\eta)$
perfil_fp.png	Gráfico do perfil $f'(\eta)$
perfil_fpp.png	Gráfico do perfil $f''(\eta)$

5 Parâmetros utilizados na execução de teste

Para validar a implementação, utilizou-se a seguinte entrada de teste:

$$s_0 = 0.3, \quad \Delta\eta = 0.01, \quad \eta_{\max} = 10, \quad (19)$$

$$\varepsilon = 10^{-10}, \quad N_{\max} = 50, \quad Re_x = 10^5. \quad (20)$$

A escolha de $\eta_{\max} = 10$ é suficiente para representar numericamente a condição distante da parede, pois o perfil $f'(\eta)$ já está praticamente estabilizado em torno de 1 nesse intervalo.

6 Resultados numéricos

6.1 Convergência do Método do Tiro

A Tabela 2 mostra a convergência do Método do Tiro para os parâmetros de teste definidos anteriormente.

Tabela 2: Histórico de convergência do Método do Tiro.

Iteração	$s = f''(0)$	$f'(\eta_{\max})$	$E(s) = f'(\eta_{\max}) - 1$
1	0.3000000000000	0.934556250742	$-6.544374925753 \times 10^{-2}$
2	0.331511994699	0.998904823862	$-1.095176137952 \times 10^{-3}$
3	0.332057188706	0.999999701862	$-2.981381648226 \times 10^{-7}$
4	0.332057337205	1.000000000000	$4.742872761199 \times 10^{-13}$

O valor convergido obtido foi

$$\boxed{f''(0) = 0.332057337205.} \quad (21)$$

O erro final foi

$$|E(s)| = 4.742872761199 \times 10^{-13}, \quad (22)$$

e o valor final de $f'(\eta_{\max})$ foi

$$f'(10) = 1.000000000000. \quad (23)$$

6.2 Perfis de similaridade

As Figuras 1, 2 e 3 apresentam os perfis obtidos numericamente.

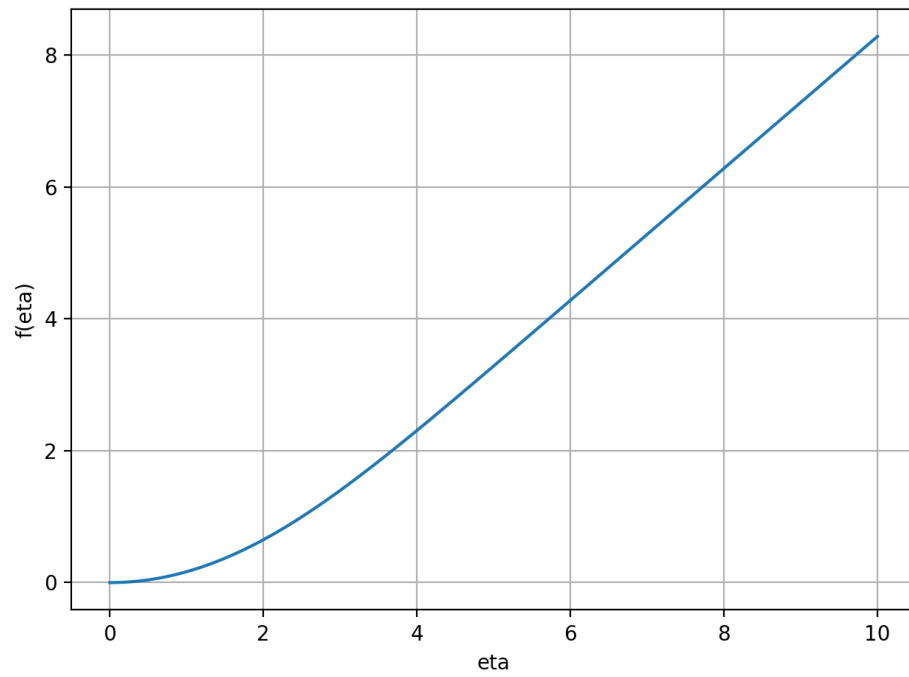


Figura 1: Perfil $f(\eta)$ obtido pela integração numérica.

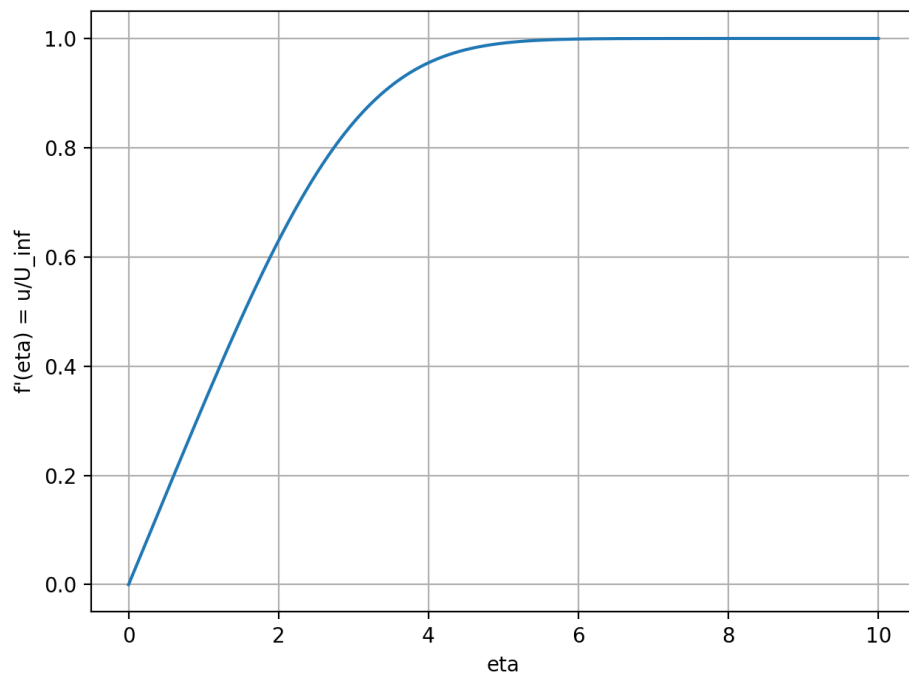


Figura 2: Perfil $f'(\eta) = u/U_\infty$. Observa-se a tendência para 1 quando η aumenta.

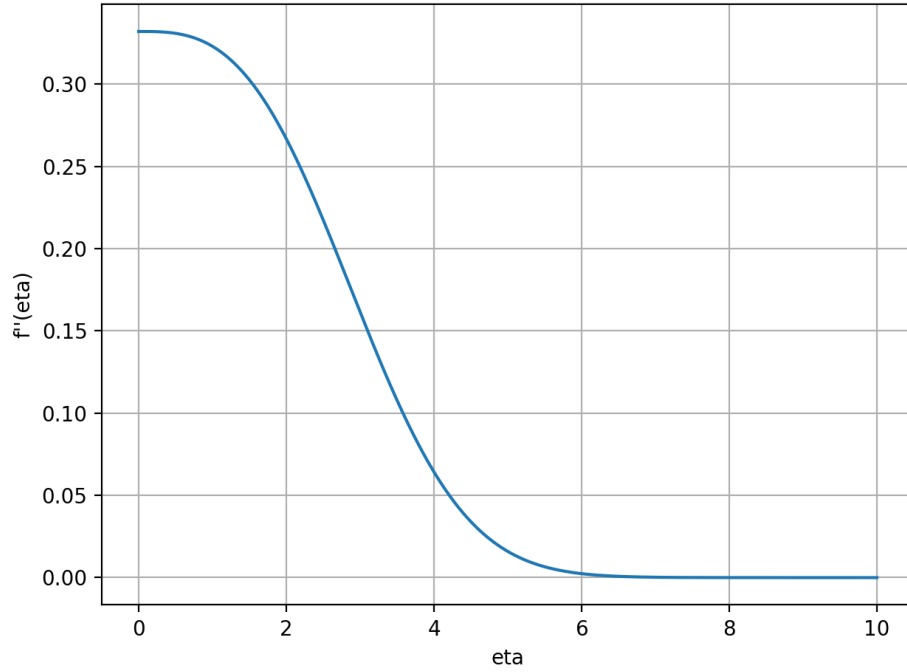


Figura 3: Perfil $f''(\eta)$, associado à variação do perfil de velocidade.

6.3 Verificação da condição distante da parede

A condição de contorno distante da parede foi verificada numericamente, pois

$$f'(\eta_{\max}) = f'(10) \approx 1.000000000000. \quad (24)$$

Assim, para o valor de $\eta_{\max}$ adotado, o perfil de velocidade já atingiu numericamente a velocidade do escoamento livre.

6.4 Coeficiente local de atrito

O coeficiente local de atrito na parede é dado por

$$C_f = \frac{2f''(0)}{\sqrt{Re_x}}. \quad (25)$$

Como

$$f''(0) = 0.332057337205, \quad (26)$$

então

$$C_f = \frac{0.664114674410}{\sqrt{Re_x}}. \quad (27)$$

Para o caso de teste $Re_x = 10^5$, obtém-se

$$C_f = 2.100114998676 \times 10^{-3}. \quad (28)$$

6.5 Determinação de η_{99} e da espessura de camada limite

A posição η_{99} foi obtida por interpolação linear no ponto em que

$$f'(\eta_{99}) = 0.99. \quad (29)$$

O valor encontrado foi

$$\boxed{\eta_{99} = 4.909989486136.} \quad (30)$$

Como

$$\delta = \eta_{99} \sqrt{\frac{\nu x}{U_\infty}}, \quad (31)$$

pode-se escrever

$$\frac{\delta}{x} = \frac{C_\delta}{\sqrt{Re_x}}, \quad (32)$$

com

$$\boxed{C_\delta = \eta_{99} = 4.909989486136.} \quad (33)$$

A correlação clássica da literatura é

$$\frac{\delta}{x} = \frac{4.92}{\sqrt{Re_x}}. \quad (34)$$

Comparando os coeficientes:

$$\frac{4.909989486136 - 4.92}{4.92} \times 100 = -0.2035\%. \quad (35)$$

Portanto, o resultado numérico obtido está muito próximo da correlação clássica.

7 Comparação com o valor clássico de $f''(0)$

O valor clássico da literatura para a solução de Blasius é

$$f''(0) \approx 0.332057. \quad (36)$$

O programa retornou

$$f''(0) = 0.332057337205. \quad (37)$$

A diferença absoluta em relação ao valor clássico arredondado é

$$|0.332057337205 - 0.332057| = 3.37205 \times 10^{-7}. \quad (38)$$

Essa diferença é compatível com a precisão esperada do método para os parâmetros adotados.

8 Fontes de erro numérico

As principais fontes de erro numérico associadas ao procedimento são:

- **Passo de integração $\Delta\eta$:** passos maiores aumentam o erro de truncamento local do método de Runge–Kutta. Embora o RK4 seja de quarta ordem, a precisão ainda depende diretamente da discretização adotada.
- **Valor de $\eta_{\max}$:** a condição real é imposta em $\eta \rightarrow \infty$, mas numericamente usa-se um valor finito. Se $\eta_{\max}$ for pequeno, o algoritmo pode forçar $f'(\eta_{\max}) = 1$ antes de o perfil estar adequadamente estabilizado.
- **Tolerância do Método do Tiro:** tolerâncias mais frouxas encerram o processo com maior erro em $f'(\eta_{\max})$.
- **Derivada numérica no Newton–Raphson:** a aproximação de $E'(s)$ por diferenças finitas introduz uma pequena dependência do valor escolhido para Δs .
- **Interpolação para η_{99} :** o valor de η_{99} é estimado por interpolação linear entre dois pontos da malha, logo também depende de $\Delta\eta$.

9 Código-fonte Python

A implementação completa é apresentada a seguir. O código solicita os parâmetros de entrada, resolve o problema pelo Método do Tiro com integração RK4, salva os arquivos de saída e gera os gráficos dos perfis.

```
1 # =====
2 # PPC5 - Calculo Numerico Aplicado
3 # Equacao de Blasius por Metodo do Tiro + RK4
4 # Autor: Mateus Leal Silva
5 # =====
6
7 import math
8
9 try:
10     import matplotlib.pyplot as plt
11 except ImportError:
12     plt = None
13
14
15 def ler_float(mensagem, padrao):
16     """Le um numero real, aceitando valor padrao e virgula decimal."""
17     texto = input(f"{mensagem} [{padrao}]: ").strip()
18     if texto == "":
19         return float(padrao)
20     return float(texto.replace(",", "."))
21
22
23 def ler_int(mensagem, padrao):
24     """Le um numero inteiro, aceitando valor padrao."""
25     texto = input(f"{mensagem} [{padrao}]: ").strip()
26     if texto == "":
27         return int(padrao)
28     return int(texto)
```

```

29
30
31 def sistema_blasius(y1, y2, y3):
32     """Retorna o lado direito do sistema equivalente de primeira ordem.
33
34     y1 = f
35     y2 = f'
36     y3 = f''
37
38     Equacao de Blasius: f''' + 0.5*f*f'' = 0
39     """
40     F1 = y2
41     F2 = y3
42     F3 = -0.5 * y1 * y3
43     return F1, F2, F3
44
45
46 def passo_rk4(y1, y2, y3, h):
47     """Executa um passo de Runge-Kutta de quarta ordem para as 3
48     variaveis."""
49     k1_1, k1_2, k1_3 = sistema_blasius(y1, y2, y3)
50     k1_1 *= h
51     k1_2 *= h
52     k1_3 *= h
53
54     k2_1, k2_2, k2_3 = sistema_blasius(
55         y1 + 0.5 * k1_1,
56         y2 + 0.5 * k1_2,
57         y3 + 0.5 * k1_3,
58     )
59     k2_1 *= h
60     k2_2 *= h
61     k2_3 *= h
62
63     k3_1, k3_2, k3_3 = sistema_blasius(
64         y1 + 0.5 * k2_1,
65         y2 + 0.5 * k2_2,
66         y3 + 0.5 * k2_3,
67     )
68     k3_1 *= h
69     k3_2 *= h
70     k3_3 *= h
71
72     k4_1, k4_2, k4_3 = sistema_blasius(
73         y1 + k3_1,
74         y2 + k3_2,
75         y3 + k3_3,
76     )
77     k4_1 *= h
78     k4_2 *= h
79     k4_3 *= h
80
81     y1_novo = y1 + (k1_1 + 2.0 * k2_1 + 2.0 * k3_1 + k4_1) / 6.0
82     y2_novo = y2 + (k1_2 + 2.0 * k2_2 + 2.0 * k3_2 + k4_2) / 6.0
83     y3_novo = y3 + (k1_3 + 2.0 * k2_3 + 2.0 * k3_3 + k4_3) / 6.0
84
85     return y1_novo, y2_novo, y3_novo

```

```

86
87 def integrar_blasius(s, deta, eta_max):
88     """Integra o PVI equivalente para um chute s = f'(0)."""
89     eta = [0.0]
90     y1 = [0.0]
91     y2 = [0.0]
92     y3 = [float(s)]
93
94     atual = 0.0
95     while atual < eta_max - 1.0e-15:
96         h = min(deta, eta_max - atual)
97         a, b, c = passo_rk4(y1[-1], y2[-1], y3[-1], h)
98         atual += h
99         eta.append(atual)
100        y1.append(a)
101        y2.append(b)
102        y3.append(c)
103
104    return eta, y1, y2, y3
105
106
107 def erro_tiro(s, deta, eta_max):
108     """Funcao erro do Metodo do Tiro: E(s)=f'(eta_max;s)-1."""
109     eta, y1, y2, y3 = integrar_blasius(s, deta, eta_max)
110     return y2[-1] - 1.0, eta, y1, y2, y3
111
112
113 def resolver_tiro(s0, deta, eta_max, tolerancia, max_iter):
114     """Atualiza o chute s por Newton-Raphson com derivada numerica."""
115     s = float(s0)
116     historico = []
117
118     for it in range(1, max_iter + 1):
119         erro, eta, y1, y2, y3 = erro_tiro(s, deta, eta_max)
120         historico.append((it, s, y2[-1], erro))
121
122         if abs(erro) < tolerancia:
123             return s, erro, it, eta, y1, y2, y3, historico
124
125         eps = max(1.0e-6, abs(s) * 1.0e-5)
126         erro_eps, *_ = erro_tiro(s + eps, deta, eta_max)
127         derivada = (erro_eps - erro) / eps
128
129         if abs(derivada) < 1.0e-14:
130             raise RuntimeError("Derivada numerica muito pequena no
131 Metodo do Tiro.")
132
133         s_novo = s - erro / derivada
134
135         # Protecao simples contra chutes nao fisicos.
136         if s_novo <= 0.0:
137             s_novo = 0.5 * s
138
139         s = s_novo
140
141     return s, erro, max_iter, eta, y1, y2, y3, historico
142

```

```

143 def localizar_eta99(eta, perfil_velocidade):
144     """Determina eta_99 por interpolacao linear em f'(eta)=0.99."""
145     for i in range(1, len(eta)):
146         if perfil_velocidade[i] >= 0.99:
147             eta_a = eta[i - 1]
148             eta_b = eta[i]
149             u_a = perfil_velocidade[i - 1]
150             u_b = perfil_velocidade[i]
151             return eta_a + (0.99 - u_a) * (eta_b - eta_a) / (u_b - u_a)
152     return None
153
154
155 def salvar_resultados(eta, y1, y2, y3, historico):
156     """Salva a solucao e o log do Metodo do Tiro."""
157     with open("blasius_solution.dat", "w", encoding="utf-8") as arq:
158         arq.write("# eta f fp fpp\n")
159         for valores in zip(eta, y1, y2, y3):
160             arq.write("{:.10f} {:.12e} {:.12e} {:.12e}\n".format(*
161                 valores))
162
163     with open("shooting_log.dat", "w", encoding="utf-8") as arq:
164         arq.write("# iter s fp_eta_max erro\n")
165         for it, s, fp_final, erro in historico:
166             arq.write(f"{it:d} {s:.12e} {fp_final:.12e} {erro:.12e}\n")
167
168 def gerar_graficos(eta, y1, y2, y3):
169     """Gera os graficos dos perfis de similaridade."""
170     if plt is None:
171         print("Matplotlib nao encontrado. Os graficos nao foram gerados.")
172         return
173
174     perfis = [
175         (y1, "f(eta)", "perfil_f.png"),
176         (y2, "f'(eta) = u/U_inf", "perfil_fp.png"),
177         (y3, "f''(eta)", "perfil_fpp.png"),
178     ]
179
180     for valores, ylabel, nome in perfis:
181         plt.figure()
182         plt.plot(eta, valores)
183         plt.xlabel("eta")
184         plt.ylabel(ylabel)
185         plt.grid(True)
186         plt.tight_layout()
187         plt.savefig(nome, dpi=200)
188         plt.close()
189
190
191 def main():
192     print("PPC5 - Equacao de Blasius por Metodo do Tiro + RK4")
193     print("Tecle Enter para usar os valores padrao sugeridos.\n")
194
195     s0 = ler_float("Chute inicial s = f''(0)", 0.3)
196     deta = ler_float("Passo de integracao Delta eta", 0.01)
197     eta_max = ler_float("Valor maximo de eta", 10.0)
198     tolerancia = ler_float("Tolerancia de convergencia", 1.0e-10)

```

```

199     max_iter = ler_int("Numero maximo de iteracoes do Metodo do Tiro",
200                        50)
201     reynolds_x = ler_float("Numero de Reynolds local Re_x", 1.0e5)
202
203     s, erro, iteracoes, eta, y1, y2, y3, historico = resolver_tiro(
204         s0, deta, eta_max, tolerancia, max_iter
205     )
206
207     eta99 = localizar_eta99(eta, y2)
208     cf = 2.0 * s / math.sqrt(reynolds_x)
209
210     salvar_resultados(eta, y1, y2, y3, historico)
211     gerar_graficos(eta, y1, y2, y3)
212
213     print("\nResultados finais")
214     print("-----")
215     print(f"f'(0) convergido           = {s:.12f}")
216     print(f"Iteracoes do tiro           = {iteracoes}")
217     print(f"Erro final                       = {erro:.12e}")
218     print(f"f'(eta_max)                     = {y2[-1]:.12f}")
219     print(f"eta_99                           = {eta99:.12f}")
220     print(f"C_delta = eta_99                = {eta99:.12f}")
221     print(f"C_f para Re_x informado = {cf:.12e}")
222     print("\nArquivos gerados:")
223     print("  blasius_solution.dat")
224     print("  shooting_log.dat")
225     print("  perfil_f.png, perfil_fp.png, perfil_fpp.png")
226
227 if __name__ == "__main__":
228     main()

```

Listing 1: Código Python utilizado para resolver a equação de Blasius.

10 Conclusão

O programa implementado resolveu a equação de Blasius sem utilizar bibliotecas de integração numérica prontas. A integração do sistema foi feita por Runge–Kutta de quarta ordem e o parâmetro desconhecido $f''(0)$ foi ajustado pelo Método do Tiro.

Para os parâmetros de teste adotados, obteve-se

$$f''(0) = 0.332057337205, \quad (39)$$

valor praticamente coincidente com o valor clássico da literatura. Além disso, o coeficiente associado à espessura de camada limite foi

$$C_\delta = 4.909989486136, \quad (40)$$

o que representa uma diferença de aproximadamente 0.2035% em relação à correlação clássica $C_\delta = 4.92$.